

# 计算机组成原理

Bowen

July 3, 2026

|          |                                   |           |
|----------|-----------------------------------|-----------|
| <b>1</b> | <b>计算机系统概述</b>                    | <b>2</b>  |
| 1.1      | 计算机的性能指标                          | 2         |
| <b>2</b> | <b>数据的表示和运算</b>                   | <b>3</b>  |
| 2.1      | 定点数的编码表示                          | 3         |
| 2.2      | 整数的表示                             | 4         |
| 2.2.1    | C 语言中的整型数据类型                      | 4         |
| 2.2.2    | 不同字长整数之间的转换                       | 4         |
| 2.3      | 运算方法                              | 4         |
| 2.3.1    | 移位运算                              | 4         |
| 2.3.2    | 标志位                               | 5         |
| 2.4      | 浮点数的表示                            | 5         |
| 2.4.1    | 理论浮点数                             | 5         |
| 2.4.2    | IEEE 754 标准                       | 6         |
| 2.4.3    | 加减运算                              | 6         |
| 2.4.4    | C 语言中的浮点数类型                       | 7         |
| <b>3</b> | <b>存储系统</b>                       | <b>7</b>  |
| 3.1      | 存储器的性能指标                          | 7         |
| 3.2      | RAM (Random Access Memory, 随机存储器) | 8         |
| 3.2.1    | DRAM                              | 8         |
| 3.3      | 多模块存储器                            | 9         |
| 3.4      | 外部存储器                             | 9         |
| 3.4.1    | 磁盘存储器                             | 9         |
| 3.4.2    | 固态硬盘                              | 10        |
| 3.5      | 高速缓冲存储器                           | 10        |
| 3.5.1    | Cache 与主存映射方式                     | 10        |
| 3.5.2    | Cache 一致性问题                       | 11        |
| <b>4</b> | <b>指令系统</b>                       | <b>12</b> |
| 4.1      | 指令的寻址方式                           | 13        |
| 4.1.1    | 指令寻址                              | 13        |
| 4.1.2    | 数据寻址                              | 13        |

# 1 计算机系统概述

| 名称   | 英文                  | 含义                                         |
|------|---------------------|--------------------------------------------|
| 存储元  | Memory Cell         | 一个二进制位 (1 bit)                             |
| 存储单元 | Storage Unit        | 一个地址对应的数据                                  |
| 存储字  | Memory Word         | 一个存储单元存放的数据 (CPU 一次存取的数据单位、存储单元存储的一串二进制代码) |
| 存储字长 | Memory Word Length  | 一个存储单元包含的二进制位数 (由编址方式决定)                   |
| 机器字长 | Machine Word Length | CPU 一次能够自然处理的二进制数据位数 (由 CPU 决定)            |

- 按地址存取方式：(主存储器的工作方式是) 按存储单元的地址进行存取
- MAR (存储器地址寄存器, Memory Address Register)：存放访存地址，位数反映最多可寻址的存储单元的个数，位数与地址线位数相同
  - 地址个数 = 终止地址 - 起始地址 + 1
  - 地址位数 =  $\log_2$  存储单元个数
    - ① 字节编址 (存储字长 1B = 8bit)：存储单元个数 = 存储器容量/1B。e.g. 主存容量 32MB，字节编址  $\Rightarrow$  地址位数 =  $2^5 * 2^{20}$
    - ② 32 位字编址 (存储字长 32bit)：存储单元个数 = 存储器容量/32bit。e.g. 主存容量 32MB，字编址  $\Rightarrow$  地址位数 =  $2^{20}$
- MDR (存储器数据寄存器, Memory Data Register)：用于暂存要从存储器中读或写的信息，位数与数据总线宽度 (位数) 相同

## Example

某 64 位计算机，主存容量 4MB，字节编址

$$\Rightarrow \begin{cases} \text{机器字长} = 64\text{bit} \\ \text{存储字长} = 1\text{B} = 8\text{bit} \\ \text{MDR 位数} = 64\text{bit} \\ \text{MAR 位数} = \log_2 \frac{4\text{MB}}{1\text{B}} = 2^{22} \end{cases}$$

- CPU 一次从主存读/写多少数据，取决于：
  - 数据总线宽度
  - 存储器接口设计
  - 一次访存操作的宽度
  - 数据总线：64bit，那么一次访存可以读：64bit = 8Byte。也就是一次可以读 8 个按字节编址的存储单元

## 1.1 计算机的性能指标

- CPU 时钟周期：CPU 工作的最小时间单位
- 主频 (CPU 时钟频率)：每秒有多少个时钟周期，单位 Hz

$$\text{主频} = \frac{1}{\text{CPU 时钟周期}}$$

3. CPI (CyclePerInstruction): 执行一条指令所需的时钟周期数

4. IPS (Instruction Per Second): 每秒执行多少条指令

$$IPS = \frac{\text{主频}}{\text{平均CPI}}$$

5. CPU 执行时间: 运行一个程序所花费的时间

$$CPU\text{执行时间} = \frac{CPU\text{时钟周期数}}{\text{主频}} = \frac{\text{指令条数} \times CPI}{\text{主频}}$$

6. MIPS (Million Instruction Per Second): 每秒执行多少百万条指令

$$MIPS = \frac{\text{指令条数}}{\text{执行时间} \times 10^6} = \frac{\text{主频}}{CPI \times 10^6}$$

7. 机器的速度与基准程序在该机器上的运行时间呈相反关系

## 2 数据的表示和运算

### 2.1 定点数的编码表示

1. 正数: 原码 == 反码 == 补码

2. 负数

- 原码  $\Leftrightarrow$  反码

- ① 符号位保留, 按位取反, 末位加 1

- ② 符号位保留, 最右 1 及右边保留, 左边取反

- $[x]_{\text{补}} \Rightarrow [-x]_{\text{补}}$

- ① 按位取反, 末位加 1

- ② 最右 1 及右边保留, 左边取反

3. 移码: 将补码的符号位取反

4. 机器数  $\Rightarrow$  真值

- 权值法

- ① 8 位补码:  $11110110 \Rightarrow -2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2^1 = -10$

- ② 16 位补码:  $8009H = 1000000000001001 \Rightarrow -2^{15} + 2^3 + 2^0 = -32759$

- ③ 32 位补码:  $FFFF8009H$  前面全是  $F$ , 说明:  $8009H$  被符号扩展到了 32 位, 所以真值和  $8009H$  相同

- 关键数

- ①  $8000H = -32768, 8001H = -32767, 8002H = -32766$

- ②  $7FFFH = 32767$

- ③  $FFFFH = -1, FFFE H = -2, FFFCH = -4$

## 2.2 整数的表示

1. 计算机中的有符号整数都用**补码**表示，所以  $n$  位有符号整数的表示范围  $-2^{n-1} \sim 2^{n-1} - 1$

### 2.2.1 C 语言中的整型数据类型

1. *short* 或 *short int*: 16bit
2. 整型 *int*: 32bit, 真值范围:  $[-2^{31} \sim 2^{31} - 1] = [-2.147483648 * 10^9 \sim 2.147483647 * 10^9]$
3. *long* 或 *long int*: 32 位机器中 32 位, 64 位机器中 64 位
4. 字符型 *char*: 8bit

### 2.2.2 不同字长整数之间的转换

1. 大字长  $\Rightarrow$  小字长: 高位部分直接截断, 低位部分直接赋值
2. 小字长  $\Rightarrow$  大字长: 高位扩展
  - 无符号整数: **零扩展**
  - 有符号整数: **符号扩展**。**符号扩展前后真值保持不变**

## 2.3 运算方法

1. 与 ( $\cdot$ ), 或 ( $+$ ), 非 ( $\bar{\phantom{x}}$ ), 异或 ( $\oplus$ , 输入相异则为 1)
2. 加法: 补码直接相加。溢出: 正 + 正 = 负 (上溢)、负 + 负 = 正 (下溢)
3. 减法: 被减数与减数的负数补码相加。溢出: 负 - 正 = 正、正 - 负 = 负
4. 乘法: 左移一位乘以 2; 加法。溢出判断
  - 有符号数: 高  $X$  位的每一位都相同, 且都等于低  $X$  位的符号, 则表示**不溢出**, 否则表示溢出
  - 无符号数: 高  $X$  位全为 0, 表示**不溢出**, 否则表示溢出
5. 除法: 右移一位除以 2; 减法。
  - 溢出判断: 若是  $2n$  位除以  $n$  位的无符号数, 商的位数为  $n + 1$  位, 当第一次试商为**1**时, 则表示结果**溢出**
  - 若是两个 32 位 *int* 型整数相除, 则除了  $\frac{\text{绝对值最大的负数}}{-1}$  会溢出, 其余情况都不会溢出
6. 无符号数和有符号数一起参与运算时, 计算机按**无符号数**来解释最终的执行结果

### 2.3.1 移位运算

1. 逻辑移位  $\Rightarrow$  无符号整数
  - 左移高位移出, 低位补 0
  - 右移低位移出, 高位补 0
  - **溢出**: 高位 1 移出, 发生溢出
  - **丢失精度**: 低位 1 移出, 丢失精度
2. 算术移位  $\Rightarrow$  有符号整数

- 左移高位移出，低位补 0
- 右移低位移出，高位补符号位
- 溢出：左移前后的符号位不同，发生溢出
- 丢失精度：低位 1 移出，丢失精度

### 2.3.2 标志位

1. 零标志 ZF:  $ZF = 1$  表示结果为 0
2. 溢出标志 OF: 判断有符号数,  $OF = C_n \oplus C_{n-1}$  (符号位进位与最高位进位的异或结果)
3. 符号标志 SF: 表示结果的符号
4. 进/错位标志 CF: 表示无符号数运算时的进位/借位
  - 加法:  $CF = 1$  结果溢出。最高位是否产生进位
  - 减法:  $CF = 1$  表示有借位，即不够减

## 2.4 浮点数的表示

### 2.4.1 理论浮点数

1. 原码尾数规格化
  - 正数:  $0.1xxx$
  - 负数:  $1.1xxx$
2. 补码尾数规格化: 符号位与最高数值位必须相反。
  - $+0.75 \Rightarrow 0.11$
  - $-0.75 \Rightarrow 1.01$

## 2.4.2 IEEE 754 标准

|                   |       | 单精度                                                                                         | 双精度                                                                                             |
|-------------------|-------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 符号 $s$            |       | 1bit                                                                                        | 1bit                                                                                            |
| 阶码 $e$            |       | 8bit(范围: [1, 254])                                                                          | 11bit(范围: [1, 2046])                                                                            |
| 尾数 $f$ (尾数有隐藏位 1) |       | 23bit                                                                                       | 52bit                                                                                           |
| 总位数               |       | 32bit (原码)                                                                                  | 64bit (原码)                                                                                      |
| 偏置值               |       | 127(7FH)                                                                                    | 1023(3FFH)                                                                                      |
| 指数                |       | 阶码真值 + 偏置值, 后按无符号整数规则转换                                                                     |                                                                                                 |
| 真值                |       | $(-1)^s \times 1.f \times 2^{e-127}$                                                        | $(-1)^s \times 1.f \times 2^{e-1023}$                                                           |
| 最小值               |       | $e = 1, f = 0 \Rightarrow 1.0 * 2^{1-127} = 2^{-126}$                                       | $e = 1, f = 0 \Rightarrow 1.0 * 2^{1-1023} = 2^{-1022}$                                         |
| 最大值               |       | $e = 254, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{254-127} = 2^{127} * (2 - 2^{-23})$ | $e = 2046, f = .111 \dots \Rightarrow 1.111 \dots 1 * 2^{2046-1023} = 2^{1023} * (2 - 2^{-52})$ |
| 阶码全 1             | 尾数全 0 | $\pm\infty$                                                                                 |                                                                                                 |
|                   | 尾数非 0 | NaN                                                                                         |                                                                                                 |
| 阶码全 0             | 尾数全 0 | $\pm 0$                                                                                     |                                                                                                 |
|                   | 尾数非 0 | 非规格化数 (隐藏位为 0)                                                                              |                                                                                                 |
| 判断溢出              |       | 规格化后阶码超出所能表示的范围时, 发生溢出                                                                      |                                                                                                 |

### 1. 真值 $\Rightarrow$ IEEE 754 浮点数

- ① 确定符号位
- ② 真值转化为二进制
- ③ 规格化确定尾数和阶码真值
  - 左规: 尾数每左移一位, 阶码减 1
  - 右规: 尾数每右移一位, 阶码加 1
- ④ 阶码真值 + 偏置值, 后按无符号整数规则转换

## 2.4.3 加减运算

### 1. 对阶: 使两个操作数的小数点对齐

- 小阶码向大阶码看齐
- IEEE 规定右移出的 *bit* 至少额外保留 3 位

## 2. 尾数加减

## 3. 尾数规格化

## 4. 舍入

- 就近舍入：0 舍 1 入。特殊情况：3 个舍弃位是 100
  - ①  $1 + 23bit$  尾数末位 0  $\Rightarrow$  截断
  - ②  $1 + 23bit$  尾数末位 1  $\Rightarrow$  截断多余位，并未位加 1
- 场景
  - ① 对阶
  - ② 右规格化

## 5. 溢出判断

- 一个正指数超过了最大允许值 (127 或 1023)  $\Rightarrow$  指数上溢 (阶码 11111111)，将尾数强制保存为全 0
- 一个负指数超过了最小允许值 (-149 或 -1074)  $\Rightarrow$  指数下溢 (若当前阶码 00000001，尾数仍需左规，直接将阶码置为全 0，并且尾数不移位)

### 2.4.4 C 语言中的浮点数类型

1. 不同类型数的混合运算，遵循的原则是类型提升，即较低类型转换为较高类型
2. *float* : 4Byte, 真值范围:  $[-3.4 * 10^{38} \sim 3.4 * 10^{38}]$
3. *double* : 8Byte, 真值范围:  $[-1.79 * 10^{308} \sim 1.79 * 10^{308}]$
4. *int*  $\Rightarrow$  *float*: 可能精度丢失
5. *double*  $\Rightarrow$  *float*: 可能溢出或精度丢失
6. *float/double*  $\Rightarrow$  *int*: 仅保留整数部分，可能溢出

## 3 存储系统

### 3.1 存储器的性能指标

1. 存储容量 = 存储字数  $\times$  存储字长
  - 存储字数：表示存储器可编址的存储单元个数 (即地址空间大小)，决定所需地址位数
    - ①  $4M$  的存储字数：即一共有  $4M = 4 * 2^{20} = 2^{22}$  个存储单元，可使用引脚复用技术，分为行、列引脚，共 11 个引脚，先后两次输入，需要 11 根地址引脚
  - 存储字长：表示一个存储单元包含的数据位数，即每个存储单元的比特数据，也表示数据引脚的数量
    - ① 按字节编址：存储容量  $4MB = 4M \times 8bit$ 。存储字长  $8bit$ ，即一个地址对应  $8bit$  数据，8 根数据引脚

② 按字编址：存储容量  $16MB = 4M \times 32bit$

③ 按半字编址：存储容量  $8MB = 4M \times 16bit$

2. 单位成本：位价 = 总成本/总容量

3. 存储速度：数据传输速率，即每秒传送信息的位数。存储速度 = 存储字长/存取周期 = 存取时间 + 恢复时间

- 存取时间 ( $T_a$ )：从启动一次存储器操作到完成该操作所经历的时间
- 存取周期 ( $T_m$ )：存储器进行一次完整的读/写操作所需的全部时间，即连续两次独立访问存储器操作之间所需的最小时间间隔
- 主存带宽 ( $B_m$ , 数据传输速率)：表示每秒从主存进出信息的最大数量

## 3.2 RAM (Random Access Memory, 随机存储器)

RAM = SRAM + DRAM

1. SRAM (Static Random Access Memory, 静态随机存储器)：即使信息被读出后，它仍保持器原状态而不需要再生（非破坏性读出）
2. DRAM (Dynamic Random Access Memory, 动态随机存储器)：Cache。必须定时刷新和读后再生
3. 主存由 RAM 和 ROM 构成

### 3.2.1 DRAM

1. 刷新周期：对同一行进行相邻两次刷新的时间间隔，通常取  $2ms$ 。e.g.  $64K \times 1bit$  的 DRAM 芯片构成  $128K \times 8bit$  的存储器，若采用异步刷新方式，每单元存储间隔不超过  $2ms$ ，则生成的刷新信号的间隔时间是？若采用集中刷新方式，则存储器刷新一遍最少用多少个读/写周期？

#### 解析

$64K \times 1bit$  由一个  $256 \times 256$  的为平面组成

构成存储器的所有芯片同时按行刷新，每个芯片有 256 行，所以存储器所有单元刷新一遍至少需要 256 次刷新操作

若采用异步刷新方式，则相邻两次刷新信息的时间间隔为  $2ms/245$

若采用集中刷新方式，则整个存储器刷新一遍至少需要 256 个读/写周期

2. DRAM 的刷新单位是行，逐行定时刷新，没刷新一行，需消耗一个存取周期
3. 每个刷新周期需刷新  $2^{\text{行地址数}}$ ，即若行地址  $6bit$ ，则每个刷新周期需刷新  $2^6 = 64$  次，消耗 64 个存取周期
4. 再生仅需恢复被读出的那些单元的数据
5. 行缓冲器：缓存指定行中每列的数据，大小 = 列数  $\times$  每列的数据位宽，常用 SRAM 实现，可支持突发传输
6. DRAM 存取周期  $>$  SRAM 存取周期  $\Rightarrow$  一次突发传输决不能跨越 DRAM 行边界



### 3.3 多模块存储器

1. 单体多字存储器
2. 多体并行存储器

- 高位交叉编址

① 连续读取  $m$  个字所需的时间  $t = mT$

- 低位交叉编址：程序连续存放在相邻模块中。可采用轮流启动或同时启动方式

① 模块的存取周期为  $T$ ，总线周期  $r$ ，存储器交叉模块数应大于或等于  $m = \frac{T}{r}$

② 连续存取  $m$  个字所需的时间  $t = T + (m - 1)r$

### 3.4 外部存储器

#### 3.4.1 磁盘存储器

1. 磁盘地址结构：驱动器号 - 柱面（磁道）号 - 盘面（磁头）号 - 扇区号

#### 磁盘的性能指标

1. 记录密度：盘片单位面积上记录的二进制数据量

- 道密度：沿磁盘半径方向单位长度上的磁道数
- 位密度：磁道单位长度上能记录的二进制代码位数
- 面密度：位密度和道密度的乘积

2. 磁盘的容量

- 非格式化容量：磁记录编码可利用的磁化单位总数。非格式化容量 = 记录面数  $\times$  柱面数  $\times$  每条磁道的磁化单位数
- 格式化容量：按照某种特定的记录格式所能存储信息的总量。格式化容量 = 记录面数  $\times$  柱面数  $\times$  每道扇区数  $\times$  每个扇区的容量
- 格式化后的容量比非格式化容量要小

3. ★存取时间 = 平均寻道时间 + 平均旋转延迟时间 + 传输时间

- 寻道时间：磁头移动到目的磁道的时间
- 平均寻道时间：从最外道移动到最内道时间的一半，和转速无关
- 旋转延迟时间：磁头定位到要读/写扇区的时间
- 平均旋转延迟时间：旋转半周的时间，和转速有关
- 传输时间：传输数据所花费的时间

4. 数据传输速率：在单位时间内向主机传送数据的字节数。假设磁盘转数为  $r$  转/秒，每条磁道容量为  $N$  字节，则数据传输速率为

$$D_r = rN = \frac{\text{每个磁道容量}}{\text{读一个磁道数据的时间}}$$

5. 读一个磁道数据的时间 = 磁盘旋转一周的时间 - 通过扇区间隙的总时间

### 3.4.2 固态硬盘

1. 数据以页为单位读/写，以块为单位擦除

## 3.5 高速缓冲存储器

1. 时间局部性：某个数据/指令在短时间内被反复访问
2. 空间局部性：最近的未来要用到的信息，很可能与现在正在使用的信息在存储空间上是临近的
3. CPU 与 Cache 之间的数据交换以字（机器字长）为单位
4. Cache 与主存之间的数据交换以 Cache 块为单位。Cache 块也称 Cache 行，每块由若干字节组成，块的长度称为块长，也称行长
5. Cache 命中率：CPU 与访问的信息已在 Cache 中的比率
6. Cache-主存系统的平均访问时间
  - $t_c$ ：命中时的 Cache 访问时间
  - $t_m$ ：未命中（缺失）时的访问时间
  - $H$ ：命中率
  - $1 - H$ ：未命中率（缺失率）
  - 串行访问： $T_a = Ht_c + (1 - H)(t_c + t_m)$
  - 并行访问： $T_a = Ht_c + (1 - H)t_m$

### 3.5.1 Cache 与主存映射方式

1. 标记项：包含有效位、脏位（修改位）、替换算法位、标记位
2. 标记位：指明 Cache 块是主存中哪一块的副本
3. 有效位：说明 Cache 行中的信息是否有效
4. 地址映射：把主存地址空间映射到 Cache 地址空间，即把存放在主存中的信息按照某种规则装入 Cache
  - 直接映射：主存中的每一块只能装入 Cache 中的唯一位置
    - ① 块冲突：若这个位置已有内容，则产生块冲突，原来的块将无条件地被替换出去
    - ② 命中：根据访存地址中间的 Cache 行号位数，找到对应 Cache 行，将对应 Cache 行中的标记与主存地址的标记位数（高位）进行比较，若相等且有效位为 1，则命中
    - ③ Cache 不存 Cache 行号和块内地址
    - ④ 计算
      - (a) 主存容量  $\Rightarrow$  主存地址位数：主存容量 = 存储字数  $\times$  存储字长，主存容量 1MB，按字节编址，存储字数 =  $1MB/1B = 2^{20}$ ，即主存地址 20bit
      - (b) Cache 有效容量：Cache 中存数据的容量，即数据块总大小
      - (c) Cache 行结构 = 有效位 + 脏位（修改位）+ 替换算法位 + Tag + 数据位（块大小 32B  $\Rightarrow$  数据位  $32 * 8\text{bit} = 256\text{bit}$ ）

- (d) Cache 容量 = (数据块 (行长) + Tag + 控制位) \* 行数/块数: Cache 数据区大小 32KB, 主存块大小 32B, 行数 =  $32KB/32B = 2^{10}$ , 即 10bit 行号
- (e) 行长/块长  $\Rightarrow$  块内地址位数: 每块  $16B = 2^4B$ , 即块内地址 4bit
- (f) 行数/块数  $\Rightarrow$  行号位数: Cache 行号位数 =  $\log_2$  Cache 行数
- (g) 主存块号 =  $\frac{\text{主存地址}}{\text{行长}}$
- (h) Cache 行号 = 主存块号 mod Cache 总行数
- (i) 主存地址结构: 标记位 - Cache 行号 - 块内地址 (与快长、行长有关)
- (j) 主存块号 = 标记位 + Cache 行号
- 全相联映射: 主存中的每一块可以装入 Cache 中的任何位置
  - ① 优点
    - (a) Cache 块的冲突概率低, 只要有空闲 Cache 行, 就不会发生冲突
    - (b) 空间利用率高
    - (c) 命中率高
  - ② 缺点
    - (a) 标记的比较速度较慢
    - (b) 实现成本较高, 通常需采用按内容寻址的相联存储器
  - ③ 主存地址结构: 标记 - 块内地址 (与快长、行长有关)
- 组相联映射: 将 Cache 分成 Q 个大小相等的组, 每个主存块可以固定组中的任意一行, 即组间采用直接映射, 组内采用全相联映射
  - ① 每组 Cache 行的数量越大, 发生块冲突的概率越低
  - ② r 路组相联: 每组中有 r 个 Cache 行
  - ③ Cache 组数 =  $\frac{\text{Cache 总容量}}{\text{Cache 块大小} \times r}$
  - ④ Cache 组号 = 主存块号 mod Cache 组数 (Q)
  - ⑤ 主存地址结构: 标记 - 组号 - 块内地址 (与快长、行长有关)
  - ⑥ 主存块号 = 标记位 + 组号
  - ⑦ 命中: 根据访存地址中间的组号找到对应的 Cache 组, 将对应的 Cache 组中每个行的标记与主存地址的高位标记进行比较, 若相等且有效位 1, 则命中
- 计算

### 3.5.2 Cache 一致性问题

#### 1. Cache 写操作命中

- 全写法 (直写法, Write Through): 当 CPU 对 Cache 写命中时, 把数据同时写入 Cache

## 和主存

- ① 写缓冲 (Write Buffer): 为减少全写法直接写入主存的时间消耗, 在 Cache 和主存之间加一个写缓冲
- ② CPU 同时写数据到 Cache 和写缓冲中, 写缓冲再将内容写入主存
- ③ 写缓冲是一个 FIFO 队列, 可以解决速度不匹配的问题, 但若出现频繁写时, 会是写缓冲饱和溢出
- 回写法 (写回法, Write Back): 当 CPU 对 Cache 写命中时, 只把数据写入 Cache, 只有当此块被替换出时才写回主存
  - ① 修改位 (脏位): 为了减少写回主存的次数, 若为 1, 则说明对应的 Cache 行中的块被修改过, 替换时须写回主存

## 2. Cache 写操作不命中

- 写分配法 (Write Allocate): 把主存块调入 Cache, 之后只修改 Cache, 搭配 Write Back, 标记 Dirty
- 非写分配法 (Not-Write Allocate): 只更新主存单元, 而不把主存块调入 Cache, 搭配 Write Through

## 3. 现代计算机使用 Write Back + Write Allocate

# 4 指令系统

## 1. ISA (指令集体系结构) 规定的内容主要包括

- 指令格式、指令寻址方式、操作类型以及每种操作对应的操作数的相应规定
- 操作数的类型、操作数寻址方式以及是按大端方式还是小端方式存放
- 程序可访问的寄存器编号、个数和位数
- 存储空间的大小和编址方式
- 指令执行过程的控制方式等, 包括程序计数器、条件码定义

## 2. 指令字长: 一条指令所包含的二进制代码的位数, 取决于操作码的长度、地址码的长度和地址码的个数。指令字长通常为字节的整数倍

- 单字长指令: 指令字长等于机器字长的指令。访存 1 次就能将指令完整取出
- 半字长指令: 指令字长等于半个机器字长的指令
- 双字长指令: 指令字长等于两个机器字长的指令。需访存 2 次才能将指令完整取出

## 3. 扩展操作码: 使操作码的长度随地址码的减少而增加, 不同地址数的指令可具有不同长度的操作码

- 设地址长度为  $n$ , 上一层留出  $m$  种状态, 下层可扩展出  $m * 2^n$  种状态

### 解析

指令字长 16bit, 4 位为基本操作码

① 13 条三地址指令  $\Rightarrow$  留出  $2^4 - 13 = 3$  种

② 40 条二地址指令  $\Rightarrow$  留出  $3 * 2^4 - 12 = 8$  种

- 不允许短码是长码的前缀
- 各指令的操作码一定不能重复

## 4.1 指令的寻址方式

寻址方式：寻找指令或操作数有效地址的方式，即确定本条指令的数据地址及下一条待执行指令的地址的方法。分为**指令寻址**和**数据寻址**两大类

### 4.1.1 指令寻址

1. 指令寻址：寻找下一条将要执行的指令地址
2. 顺序寻址：通过程序计数器 PC 加 1 (**1 条指令的长度**)，自动形成下一条指令的地址
3. 跳跃寻址：由本条指令给出下条指令地址的计算方式。通过转移类指令实现
  - 绝对转移：地址码直接指出转移目标地址
  - 相对转移：地址码指出转移目的地址相对于当前 PC 值的偏移量

### 4.1.2 数据寻址

1. 数据寻址：如何在指令中表示一个操作数的地址，或怎样计算出操作数的地址
2. 寻址特征：指明属于哪种寻址方式
3. 形式地址 ( $A$ )：指令中的地址码字段并不代表操作数的真实地址
4. 有效地址 ( $EA$ )：形式地址结合寻址方式，可以计算出操作数在存储器中的真实地址，即  $EA$

|         | 含义                                                                                 | 优点                                                                           | 缺点                                   | 备注                             |
|---------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------|--------------------------------|
| 隐含寻址    | 不明显地给出操作数的地址，而是隐含操作数的地址                                                            | 有利于缩短指令字长                                                                    | 需要增加存储操作数或隐含地址的硬件                    |                                |
| 立即（数）寻址 | 指令字中的地址字段指出的是操作数本身，也称 <b>立即数</b> ，采用 <b>补码</b> 表示                                  | 指令在执行阶段不访存，指令执行速度最快                                                          | $A$ 的位数限制了立即数的范围                     | 特征：#                           |
| 直接寻址    | 指令字中的形式地址 ( $A$ ) 就是操作数的真实地址 ( $EA$ )                                              | 简单，指令在执行阶段仅需访存一次                                                             | $A$ 的位数限制了该指令操作数的寻址范围，操作数的地址不易修改     |                                |
| 间接寻址    | 指令的地址字段是操作数有效地址所在主存单元的地址，即操作数地址的地址                                                 | 可扩大寻址范围，便于编制程序（用间接寻址可方便地完成子程序返回）                                             | 指令执行阶段要多次访存（ $n$ 次间接寻址需 $n + 1$ 次访存） |                                |
| 寄存器寻址   | 指令字中的地址字段给出的是操作数所在寄存器的编号                                                           | 执行阶段不用访存，执行速度快；寄存器数量远小于内存单元数，所以地址码位数较少，指令字长较短                                | CPU 的寄存器数量有限，寻址能力有限                  |                                |
| 寄存器间接寻址 | 指令字中的 $R_i$ 所指寄存器给出操作数所在主存单元的地址                                                    | 扩大了寻址范围，减少了访存次数                                                              |                                      |                                |
| 相对寻址    | 将 PC 的内容加上指令格式中的形式地址 ( $A$ ) 而形成操作数的有效地址，其中 ( $A$ ) 是相对于下一条 PC 值的偏移量， <b>补码</b> 表示 | 操作数的地址随 PC 值的变化而变化，且与指令地址之间总是相差一个固定的偏移量，因此便于程序浮动（ <b>一段代码在程序内部的浮动</b> ）      |                                      | 广泛应用于 <b>转移指令</b>              |
| 基址寻址    | 将基址寄存器 ( $BR$ ) 的内容加上指令字中的形式地址 ( $A$ ) 而形成操作数的有效地址                                 | 扩大寻址范围（基址寄存器的位数大于形式地址 ( $A$ ) 的位数）；有利于多道程序设计，并可用于编制浮动程序（ <b>整段代码在内存中浮动</b> ） |                                      | 面向系统，用于为多道程序或数据分配存储空间          |
| 变址寻址    | 将变址寄存器 ( $IX$ ) 的内容加上指令字中的形式地址 ( $A$ ) 而形成操作数的有效地址                                 | 扩大寻址范围；适合编制循环程序                                                              |                                      | 立足于用户，主要用于处理数组问题，变址寄存器的内容有用户设定 |
| 堆栈寻址    |                                                                                    |                                                                              |                                      |                                |